

Architecture & Ecosystem Design — Part 4:

Monorepo & Repository Architecture

1. Repository Philosophy

The repository is the physical representation of the AROH architecture. It should communicate system boundaries through its structure. Every folder, package, application, and configuration should clearly indicate ownership, responsibility, and dependency direction.

The repository must optimize for:

Scalability

Maintainability

Discoverability

Developer Experience

AI-assisted Development

Independent Deployment

Shared Development

Long-term Evolution

Consistency

Minimal Duplication

Every directory should exist because it has a well-defined architectural purpose. No folder should become a dumping ground for unrelated files. Repository organization is considered part of system design, not merely file management.

2. Monorepo Philosophy

AROH adopts a monorepo because every product belongs to the same ecosystem.

The goal is not to place everything into one repository.

The goal is to centralize:

Shared Code

Shared Standards

Shared Dependencies

Shared Tooling

Shared Documentation

Shared Packages

Shared Configuration

Shared Infrastructure

Shared Testing

Shared Development Workflow

while preserving product independence.

The monorepo should feel like multiple software products sharing one engineering platform.

3. High-Level Repository Structure

```
aroh/  
  
apps/  
packages/  
platform/  
tooling/  
docs/  
scripts/  
configs/  
assets/  
.github/
```

Every top-level directory has one responsibility.

Nothing exists outside these architectural boundaries.

4. Apps Directory

The Apps directory contains deployable applications.

Examples

```
apps/  
  
aroh  
nebula  
javapath-pro  
spedex  
music-mirror  
task-pro  
admin  
developer-portal  
future-products
```

Rules

Every app has:

Own Routes

Own Components

Own Assets

Own Tests

Own Documentation

Own Configuration

Own Business Logic

Apps never duplicate platform services.

Apps never communicate directly.

Apps consume Platform APIs.

Apps remain independently deployable.

5. Platform Directory

Platform contains reusable business capabilities.

Example

```
platform/  
  
identity  
wallet  
membership  
notifications  
analytics  
ai  
registry  
configuration  
search  
billing  
monitoring  
audit  
feature-flags  
media  
localization
```

Platform modules own business infrastructure.

Products consume.

Platform evolves.

Products remain stable.

6. Packages Directory

Packages contain reusable development libraries.

Example

```
packages/  
  
ads
```

```
asdk
motion
icons
ui
utils
eslint-config
tsconfig
shared-types
```

Packages should never contain product logic.

Packages are reusable.

Versioned.

Documented.

Tested.

Framework-agnostic whenever practical.

7. Documentation Directory

Documentation evolves with implementation.

Structure

```
docs/

architecture
adr
development
deployment
api
security
testing
design
product
operations
reference
```

Documentation should follow code.

Code should follow architecture.

Neither should drift independently.

8. Tooling Directory

Engineering tooling belongs here.

Examples

Code Generation

Templates

CLI

Migration Tools

Automation

Quality Scripts

Dependency Analysis

Architecture Validation

AI Prompt Frameworks

Release Tools

Build Helpers

Tooling should improve engineering productivity.

9. Assets Directory

Shared assets.

Logos

Branding

Icons

Videos

Fonts

Illustrations

Media

Motion Assets

Marketing Resources

Products should reference assets.

Not duplicate them.

10. Configuration Directory

Central configuration.

TypeScript

ESLint

Prettier

Tailwind

Vitest

Jest

Playwright

CI

Deployment

Workspace

Environment

Aliases

Build Configuration

Shared configs prevent configuration drift.

11. Scripts Directory

Automation only.

Environment Setup

Build

Deployment

Migration

Database

Seeding

Validation

Release

Backup

Maintenance

Scripts should be repeatable.

Safe.

Documented.

12. Dependency Rules

Dependencies always flow downward.

Apps

↓

Platform

↓

Packages

↓

Infrastructure

Never reverse.

Never circular.

Never lateral between products.

13. Naming Standards

Directories

lowercase-kebab-case

Packages

@aroh/package-name

Components

PascalCase

Hooks

useHookName

Utilities

camelCase

Interfaces

PascalCase

Types

PascalCase

Constants

UPPER_SNAKE_CASE

Files

Consistent.

Predictable.

Descriptive.

Avoid abbreviations.

14. Package Architecture

Every package contains:

```
src/  
  
components  
hooks  
types  
utils  
constants  
providers  
services  
styles  
tests
```

Optional

examples

stories

benchmarks

migrations

Documentation accompanies every public package.

15. Platform Module Architecture

Each module follows one structure.

```
identity/  
  
application/  
domain/  
infrastructure/  
api/  
contracts/  
events/  
tests/  
docs/
```

Application

Use Cases

Commands

Queries

Domain

Entities

Policies

Rules

Infrastructure

Firebase

Database

Providers

External APIs

Contracts

Interfaces

DTOs

Events

Domain Events

Integration Events

16. Product Architecture

Every product should mirror the same layout.

```
app/  
  
components  
features  
services  
hooks  
pages  
assets  
tests  
docs
```

Developers should recognize every project immediately.

17. Shared Design System Architecture

ADS should eventually contain

Typography

Buttons

Inputs

Forms

Cards

Tables

Navigation

Charts

Layouts

Dialogs

Modals

Icons

Colors

Spacing

Grid

Responsive Helpers

Accessibility Helpers

Motion Tokens

Theme Tokens

Everything visual belongs here.

18. ASDK Architecture

ASDK should provide

Schemas

Validation

API Clients

Authentication Helpers

Wallet Helpers

Configuration

Utilities

Shared Hooks

Shared Stores

Types

Permission Helpers

Logging

Error Models

Testing Utilities

Migration Helpers

Future SDK Plugins

Products import ASDK.

Never duplicate helpers.

19. Infrastructure Abstraction

Products should never know:

Firestore

Supabase

MongoDB

REST

GraphQL

Storage Provider

Authentication Provider

Notification Provider

Analytics Provider

Everything passes through abstractions.

Infrastructure replacement should affect Platform modules only.

20. Environment Strategy

Development

Testing

Staging

Production

Future Preview

Every environment has

Own Variables

Own Secrets

Own Services

Own Monitoring

Own Logging

No hardcoded configuration.

21. Versioning Strategy

Platform

Independent Version

Packages

Independent Version

Products

Independent Version

Documentation

Versioned

Database

Migration Version

APIs

Versioned

Breaking changes documented.

Migration required.

22. Build Strategy

Incremental Builds

Shared Cache

Workspace Awareness

Parallel Compilation

Package Reuse

Independent Deployments

Selective Testing

Architecture Validation

Dependency Analysis

Fast CI

Future distributed builds remain possible.

23. Repository Governance

Every change must answer

Why?

Who owns it?

Can it be reused?

Does it belong in Platform?

Does it belong in Product?

Does it introduce duplication?

Does it violate architecture?

Can another developer understand it?

Can AI understand it?

Will this scale?

Architecture reviews occur before implementation reviews.

24. Future Repository Evolution

As AROH expands, the repository should support dozens of applications, platform modules, packages, and engineering tools without becoming difficult to navigate. The repository itself should communicate architectural intent. A developer joining the project should understand ownership, dependencies, extension points, and system boundaries simply by reading the directory structure. The repository is therefore treated as an architectural artifact, not merely a storage location for source code. Every new folder, package, module, and configuration should strengthen the ecosystem rather than increase complexity, ensuring the repository remains scalable, maintainable, and aligned with the long-term vision of AROH as a platform-first software ecosystem.